Moodle's Architecture

Assignment #3

Software Systems Architecture

1MEIC02 T3

Gonçalo Gonçalves da Costa Pinto
José Diogo Alves Granja
Manuel Mo
Pedro Rafael Correia Borges
Ricardo Miguel Yang

1. Moodle

Moodle is an open-source Learning Management System (LMS) written in PHP. It provides an online environment where teachers and students interact through courses, which contain resources (such as files, pages, and links) and activities (such as forums, quizzes, and wikis).

From an architectural standpoint, Moodle has a monolithic, but also plugin-based modular design. One of its defining characteristics is its relatively “fat core”, which means that unlike in a pure microkernel architecture, where the core would consist of a simple, lightweight plugin-loader stub with just about everything else being plugins, Moodle’s core contains a massive amount of base functionalities.

2. Core Components and Layers

Moodle’s architecture is essentially separated into three distinct physical parts and logical layers. Physically, its installation is divided between the code, the database, and the moodledata. The **code** consists of PHP scripts executed by the web server, which for security reasons cannot be writable by the server. The **database** is managed by one of the supported RDBMSs, such as MySQL or PostgreSQL, utilizing an abstraction layer to ensure compatibility by adding a prefix to all the table names. Finally, the **moodledata** is a secure folder located outside the web root where Moodle stores user-uploaded and generated files, making it the only physical component that needs to be writable by the web server.

Logically, the system utilizes a Layered Architecture, divided into a presentation layer, an application layer, and a data layer, as shown in Figure 1. The **presentation layer** is managed by themes and global UI objects that dictate the layout and appearance. The **application layer** consists of the “fat core” logic and the extensive plugin ecosystem. And finally, the **data layer**, which is handled by the database abstraction and the moodledata file system. This way, it enforces a strict separation between data access mechanisms, core logic, and presentation rendering.

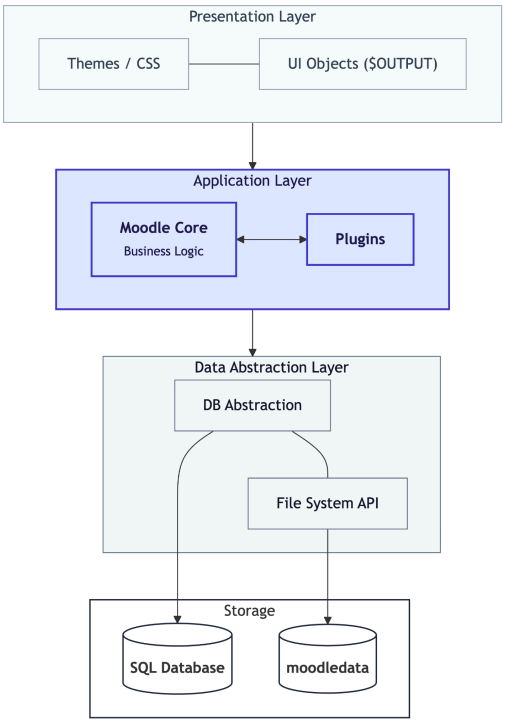

Figure 1: Moodle’s Logical Architectural Layers

2.1 Request Dispatching

Rather than the now common single entry point router, Moodle routes requests using the traditional PHP approach to URLs, where identifiers (IDs) remain visible in the query string, rather than being hidden behind semantic or user-friendly URLs.

For example, viewing a course page is handled directly by a URL pointing to a specific script, such as *.../course/view.php?id=123*.

While this design decision results in URLs that are less aesthetically pleasing from the end-user's perspective, it prioritizes usability for developers, who can instantly identify and understand how a specific request is handled. Furthermore, it ensures that the URLs act as permanent links that remain stable even if the course names change, guaranteeing stability and trust for the users over time.

2.2 The Plugin System and Frankenstyle

As discussed before, Moodle consists of a “fat core” that acts as a central orchestrator, surrounded by over 35 different types of strongly-typed plugins used to extend or add functionalities (represented in Figure 2). Being strongly-typed means that, depending on the functionality needed, developers must write a different type of plugin and interact with the core via a specialized API (such as the Activity or Authentication API), contrasting with architectures where all plugins use a single, generic API.

To organize its massive number of plugins, Moodle uses a naming convention called “Frankenstyle”, which combines the plugin type and its name. For example, *mod_forum* is the Frankenstyle for an activity module named forum. This name maps directly to the file system where the type serves as the prefix directory and the name as the folder name, resulting in the path *mod/forum*. For functionalities that do not fit into these specific categories, Moodle provides a local plugin type as a catch-all for miscellaneous features.

Each plugin contains a *version.php* file with metadata for automatic installation and upgrades, and a *db/access.php* file to define permissions. It is also important to note that only Activity module plugins (the mod type) have the special privilege of supporting sub-plugins. This constraint prevents the severe performance degradation that would occur if all plugin types were allowed to load their own sub-dependencies.

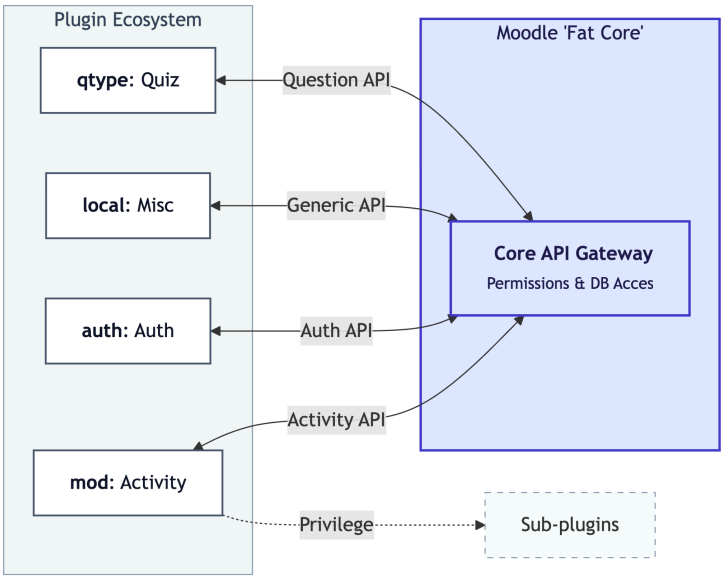

Figure 2: Plugin Ecosystem and “Fat Core” Interaction

3. Key Architectural Patterns

Moodle's architecture relies on several well-known patterns to manage the complexity of its codebase, with the most prominent one being the **Plugin-Based Architecture**. As established earlier, Moodle consists of a heavyweight core, with the vast majority of its educational features added via plugins. It is often analyzed through the lens of the **Microkernel pattern**, as it shares that pattern's central hub-and-spoke structure; however, it adapts this concept by featuring a 'fat core' rather than a minimal one. This approach effectively creates a **Modular Monolith**, where the system provides standard APIs and essential services at its center, while entirely self-contained plugins hook into this core to provide specific features.

Another foundational pattern present is the **Model-View-Controller (MVC) pattern**, specifically implemented through the **Page Controller** pattern to fit an older PHP style, as shown in Figure 3. Firstly, the individual PHP scripts, such as index.php or view.php, act as the Page Controllers that process incoming HTTP requests and execute business logic. Then, the Data Access/XMLDB layer serves as the Model, handling database interactions. Finally, the View is abstracted through global objects like $PAGE, which stores contextual information about the page, and $OUTPUT, which generates the final HTML headers, boxes, and footers.

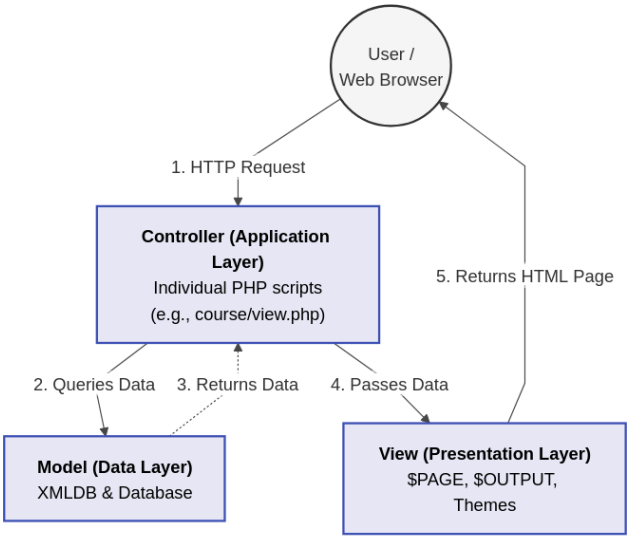

Figure 3: Moodle's MVC implementation using Page Controller

To tie these components together without constantly passing variables through function parameters, Moodle also implements the **Registry** pattern. During the initial bootstrap phase triggered by *config.php*, several global objects are instantiated, including $CFG for system configuration, $USER for the current session, and $DB for the database connection. These global registries are then reliably available to any script or plugin operating within the system.

4. Quality Attributes and Trade-offs

Moodle's design has many quality attributes that are essential for its success as an open-source educational platform, although some of them come with deliberate trade-offs.

First, let's start with its most important attribute: **Modifiability**. Each educational institution has its very own specific needs, and Moodle's strongly-typed plugin system allows administrators to drastically change, extend, and tailor the platform to those needs without having to touch the core code itself. It's via this degree of **isolation** that Moodle supports **modifiability** alongside **maintainability**, as the core system can go through updates without breaking the institution's custom plugins.

Portability and **Security** are also major focal points of Moodle's design. Portability is achieved by relying on PHP and a custom database abstraction layer, which ensures that Moodle can be deployed on a wide variety of hardware platforms and database systems, accommodating the diverse IT budgets of different universities. Security is then managed through a multi-layered Contexts and Capabilities system: since educational records require strict privacy, this system allows administrators to define granular permissions that cascade down from the site level to individual courses and specific activities, protecting against unauthorized access.

However, for some features to be achievable some compromises were necessary, for example, with the evolution of the "fat core" discussed previously on this assignment. While the developers' long-term architectural goal has been to improve maintainability by shrinking the core and moving functionality into plugins, the constant demand for new, standardized features continuously expands it, thus creating a perpetual tension between keeping the system lightweight and providing a proper out-of-the-box functional platform.